Python compact cheatsheet

STEP 1 Core Python · Getting started

1.1 What a program is and your first Python program

Program and language A program is a list of exact instructions. Lexis is the vocabulary, syntax the grammar, semantics the meaning.

How CPython runs a file Source is checked and compiled to bytecode, then the virtual machine runs it top to bottom. A SyntaxError anywhere stops everything.

print() print(*values, sep=" ", end="\n"). str() on each value, sep between, end after. N = L + (n − 1)s + e.

Keywords and blocks 35 keywords in 3.14, soft keywords match, case, type, _. # starts a comment. Indent blocks with 4 spaces.

Three errors Syntax error before running, runtime error at the bad line, logic error never reported.

The full form of print()
print(*values, sep=" ", end="\n", file=sys.stdout, flush=False)

Characters written by one print()
$N = \sum \text{len}(\text{str}(v_i)) + (n - 1) \cdot \text{len}(\text{sep}) + \text{len}(\text{end})$

Characters from one print()
$N = L + (n - 1) \cdot s + e$

TRAP In the REPL, typing 2 + 3 shows 5 without print(). In a .py file, nothing appears unless you call print(). Beginners often expect the REPL behaviour inside a file.

EXAM sep and end must be given by name (keyword arguments). print("A", "B", "-") prints A B - with the dash as a third value, it does not change sep.

TRAP Python uses indentation where many languages use curly brackets. Mixing tabs and spaces, or using 3 spaces on one line and 4 on the next, gives IndentationError. PEP 8 says: always 4 spaces.

EXAM PCEP asks which names follow PEP 8 conventions. snake_case for variables and functions, CapWords for classes, UPPER_CASE for constants.

1.2 Variables, values, types and numeral systems

Names and objects name = value binds a name to an object. b = a gives a second name to the same object. Objects have id, type, value.

Basic types int (unlimited), float (about 15 to 17 digits), complex, bool (subclass of int), str, None. All immutable.

Bases 0b, 0o, 0x literals. bin(), oct(), hex() to text. int(text, base) back. N = sum of $d_i\, b^i$.

Conversions int() cuts towards zero, int("3.5") is ValueError, round() ties to even, bool("0") is True.

Naming Letters, digits, _ , no leading digit, no keyword, case sensitive, snake_case, do not shadow built ins.

Scientific notation $a\ e\ b = a \times 10^b$

Value of a number written in base b
$\text{value} = d_k b^k + \ldots + d_1 b^1 + d_0 b^0$

Base b to decimal $N = \sum_{i=0}^{k} d_i \cdot b^i$

Decimal to base b
$d_0 = N \bmod b,\ N \leftarrow N \,//\, b$, repeat

EXAM bool is a kind of int: True == 1 and False == 0, so True + True is 2. This surprises many people in exams.

TRAP A leading zero is not allowed on a decimal literal in Python 3: 017 is a SyntaxError. Write 0o17 for octal.

EXAM int(-3.99) is -3, not -4. int() cuts towards zero. math.floor(-3.99) is -4 because floor goes down.

TRAP Naming a variable list or str hides the built in. After list = [1, 2], calling list("abc") fails with TypeError: 'list' object is not callable.

1.3 Operators, expressions and the order Python works them out

Division / gives float. // floors (towards minus infinity). % has the sign of the divisor. a == (a // b) * b + a % b.

Priority brackets, **, unary, * / // %, + -, shifts, & ^ |, comparisons, not, and, or. ** right to left. -2 ** 2 = -4.

Comparisons Chained 1 < x < 10. Floats: use math.isclose(). 0.1 + 0.2 == 0.3 is False.

Logic and, or short circuit and return an operand. not > and > or.

Bits ~x = -x - 1. x << k = x * 2**k. x & 1 tests odd. x & (x - 1) == 0 tests power of 2.

Floor division $a \,//\, b = \lfloor a \div b \rfloor$

Remainder (modulo) $a \bmod b = a - b \cdot (a \,//\, b)$

The division identity $a == (a \,//\, b) * b + a \bmod b$

Safe float comparison
$|a - b| \le \max(\text{rel} \cdot \max(|a|, |b|), \text{abs_tol})$

Bitwise not $\sim x = -x - 1$

Shifts $x << k = x \cdot 2^k$, $x >> k = x \,//\, 2^k$

Euclidean style division in Python
$a = q \cdot b + r,\ q = \lfloor a/b \rfloor,\ 0 \le r < b\ (b > 0)$

Last digit and dropping it $n \bmod 10,\ n \,//\, 10$

EXAM A unary operator has one operand (-x, not x, ~x). A binary operator has two (a + b). The same symbol - can be either.

EXAM C and Java cut integer division towards zero, so -7 / 2 is -3 there. Python floors, so -7 // 2 is -4. Exams love this difference.

TRAP -2 ** 2 is -4, not 4. The power is done first, then the minus. Write (-2) ** 2 when you mean minus two squared.

EXAM not has a higher priority than and, and and is higher than or. So True or False and False is True or (False and False) = True.

TRAP "5" + 5 is a TypeError. Python never turns a number into text by itself. Use str(5) or an f-string.

1.4 Input, output and formatting text

input() Shows a prompt, waits, returns text. Convert with int() or float(). a, b = map(int, input().split()).

f-strings f"{expr}", {x!r}, {x = }, {x:spec}. {{ and }} for real brackets.

Format spec [[fill]align][sign][#][0][width][grouping][.precision][type]. Width is a minimum. .2f rounds half to even.

Older formatting "%s %d" % (a, b) and "{} {}".format(a, b). Prefer f-strings.

Safe input while True + try int(text) except ValueError + range check + break.

Format spec grammar
[[fill]align][sign][#][0][width][grouping][.precision][type]

Padding added by a width
$\text{pad} = \max(0, \text{width} - \text{len}(\text{text}))$

Centre alignment
$\text{left} = \lfloor \text{pad} / 2 \rfloor,\ \text{right} = \text{pad} - \text{left}$

TRAP input() returns text even when the user types digits. "5" + "5" is "55". Convert first, then calculate.

EXAM .2f rounds the stored binary value, and exact halves go to the even digit: f"{2.5:.0f}" gives 2 and f"{2.675:.2f}" gives 2.67.

STEP 2 Core Python · Control flow

2.1 Making decisions: if, elif, else and match

Chains if, any number of elif, optional else. First truthy condition wins, the rest are skipped. Strictest test first.

Truthiness Falsy: False, None, 0, 0.0, "", empty collections. "0" and [0] are truthy.

Combining and, or, not, 13 <= x <= 19, x in (a, b). x == a or b is always truthy.

One liners a if cond else b. (n := len(x)) assigns inside an expression.

match Literal, capture, _, |, [a, *rest], {"k": v}, Class(), guard if. A bare name captures.

Conditional expression
value_if_true if condition else value_if_false

De Morgan laws
not (a and b) == (not a) or (not b), not (a or b) == (not a) and (not b)

Number of paths
k independent if statements give up to 2^k combinations

TRAP if day == "Sat" or "Sun": is always True. Python reads it as (day == "Sat") or "Sun", and "Sun" is truthy. Write day in ("Sat", "Sun").

TRAP A plain name in a case is a capture, not a comparison. case RED: catches everything and binds it to RED. To compare with a constant, use a dotted name such as case Color.RED:.

2.2 Loops: while, for, range, break and continue

while and for while repeats while a condition is True. for repeats once per item of an iterable.

range range(start, stop, step) excludes stop. len = max(0, ceil((stop - start)/step)). step 0 is an error.

Loop control break leaves, continue skips, pass does nothing, loop else runs when no break happened.

Patterns Accumulator, counter, running max, search with break, build a list. Nested loops: outer x inner turns.

Pythonic loops enumerate(items, start=1), zip(a, b) stops at the shortest, reversed(), sorted().

range(start, stop, step)
start, start + step, start + 2·step, ... while still before stop

How many numbers range gives
$\text{len} = \max(0, \lceil(\text{stop} - \text{start}) / \text{step}\rceil)$

Turns of a nested loop outer × inner

Triangle loop $1 + 2 + \ldots + n = n(n + 1)/2$

Length of a range
$\text{len}(\text{range}(a, b, s)) = \max(0, \lceil(b - a)/s\rceil)$

Sum of an arithmetic sequence
$S = n(\text{first} + \text{last})/2$

EXAM range(5) is 0, 1, 2, 3, 4: five numbers, but 5 itself is not included. This "stop is excluded" rule matches list indexes, so range(len(items)) gives every valid index.

EXAM The loop else is not "else if the loop did not run". It means "no break happened". A for loop over an empty list runs its else block.

TRAP break only leaves the inner loop. To leave two loops at once, put them in a function and use return, or set a flag and check it in the outer loop.

TRAP zip stops at the shortest list. zip([1, 2, 3], "ab") gives only two pairs. Python 3.10 added zip(..., strict=True) to raise an error when the lengths differ.

STEP 3 Core Python · Data collections

3.1 Strings: text, slicing, methods and Unicode

Making strings Quotes: single, double or triple. Escapes start with a backslash. r"..." is a raw string. len() counts characters.

Index and slice s[0] first, s[-1] last. s[a:b:c] includes a, excludes b. s[::-1] reverses. Slices never raise IndexError.

Immutable You cannot change a character. Methods return new strings: write s = s.upper().

Key methods strip, split, join, replace, find (-1) vs index (ValueError), count, startswith, isdigit, isalpha, upper, lower.

Unicode ord("A") = 65, chr(97) = "a". UTF-8 uses 1 to 4 bytes per character. Compare strings by code point.

Index of a character
0, 1, ..., len(s) − 1 from the left, and −1, −2, ..., −len(s) from the right

Slice s[start:stop:step]

Number of substrings of a text of length n
$n(n + 1) / 2$

Number of different arrangements (anagrams)
$n! / (c_1! \cdot c_2! \cdot \ldots)$

EXAM A backslash starts an ESCAPE SEQUENCE : the backslash and the next character together stand for one special character. That is why len of the new line escape is 1. In a raw string (r before the quote) Python keeps every backslash, which is handy for Windows paths and regular expressions (Advanced 6.1).

EXAM A single index outside the string raises IndexError. A slice outside the string never fails: it just gives what is there, or an empty string.

TRAP Strings are compared letter by letter by code point, not by dictionary order. All upper case letters (65 to 90) come before all lower case letters (97 to 122), so "Zebra" < "apple" is True. "10" < "9" is True too, because "1" comes before "9". Turn text into numbers before comparing numbers.

TRAP Every method returns a new string. s.upper() alone on a line does nothing useful. Write s = s.upper() to keep the result.

EXAM split() with no argument and split(" ") are different. "a b".split() gives ["a", "b"], but "a b".split(" ") gives ["a", "", "b"] because there are two spaces.

TRAP Always say the encoding when you open a text file: open(name, encoding="utf-8"). Otherwise Python uses the system default, which is not UTF-8 on every Windows computer (Core 5.3).

3.2 Lists: storing, changing, sorting and copying

Lists Ordered, changeable, any types. Index from 0, slice like strings. len, min, max, sum, in.

Add and remove append (one item), extend (many), insert(i, x). remove(x) first match, pop() last, pop(i), del items[i].

Sorting sort() changes the list and returns None. sorted() returns a new list. key= and reverse=. Stable.

Copies b = a is the same list. a[:] or a.copy() is a shallow copy. copy.deepcopy for nested lists. [[0]*n for _ in range(m)].

Costs append, pop(), index access O(1). insert(0), pop(0), in, remove O(n). Bubble sort n(n-1)/2 comparisons.

Item of a matrix matrix[row][col]

Comparisons in the worst case
$(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2$

Average cost of an append (amortised)
(n + total copies) / n < a small constant

Bubble sort comparisons $n(n - 1)/2$

EXAM append adds one item, even if that item is a list. [1, 2].append([3, 4]) gives [1, 2, [3, 4]]. extend adds the items one by one and gives [1, 2, 3, 4].

TRAP Do not remove items from a list while a for loop is walking over it. The loop skips the item after each removal. Loop over a copy (for x in items[:]) or build a new list instead.

EXAM [[0] * 3] * 3 makes one inner list and three references to it, so changing one row changes all three. Build a grid with a comprehension: [[0] * 3 for _ in range(3)].

EXAM The number of names must match the number of items, unless one name has a star. a, b = [1, 2, 3] raises ValueError: too many values to unpack.

3.3 Tuples and sets: fixed records and unique items

Tuples Immutable ordered sequence. (5,) is a tuple, (5) is not. count and index only. Unpack with a, b = t.

Hashable Dict keys and set items must be hashable: numbers, strings, tuples of hashables, frozensets. Not lists, sets or dicts.

Sets Unique items, no order. set() is empty. add, remove (KeyError), discard (safe), pop (any item).

Set algebra | union, & intersection, - difference, ^ symmetric difference, <= subset. Methods accept any iterable.

Counting |A u B| = |A| + |B| - |A n B|. A set of n items has 2^n subsets. x in set is O(1) on average.

Size of a union (inclusion and exclusion)
$|A \cup B| = |A| + |B| - |A \cap B|$

Number of subsets (the power set) 2^n

EXAM The comma makes a tuple, not the brackets. (5) is the number 5. (5,) is a tuple with one item. This is a favourite exam trick.

TRAP A tuple holds references. You cannot change which objects it holds, but if one of them is a list, that list can still change. Such a tuple is not hashable, so it cannot be a dict key.

EXAM {} is an empty dict, not an empty set. Write set() for an empty set. Also, a set can only hold hashable items: numbers, strings, tuples of them, but not lists or other sets.

EXAM The operators need two sets. The methods accept any iterable, so A.union([1, 2]) works but A | [1, 2] raises TypeError.

3.4 Dictionaries: keys, values and hash tables

Dicts Key to value mapping. Keys unique and hashable, values anything. Ordered by insertion since 3.7.

Read safely d[k] raises KeyError. d.get(k, default) does not. k in d checks keys only.

Change d[k] = v adds or replaces. del d[k], d.pop(k, default), d.popitem() (last added), update, |, |=.

Loop for k in d (keys), d.values(), d.items(). Do not add or remove keys while looping, use list(d).

Patterns Count: d[k] = d.get(k, 0) + 1 or Counter. Group: dict of lists or defaultdict(list). Two Sum: dict of seen values.

Slot of a key slot = hash(key) mod table size

Load factor α = used slots / table size ≤ 2/3

EXAM for x in d gives the keys, not the values and not the pairs. Use d.values() for values and d.items() for (key, value) pairs. x in d also checks keys only.

TRAP keys(), values() and items() return VIEWS : windows onto the dict that update by themselves. Adding or removing keys while you loop over a dict raises RuntimeError. Loop over list(d) instead.

TRAP dict.fromkeys(keys, []) gives every key the same list object. Appending to one changes all of them. Use a comprehension {k: [] for k in keys} instead.

EXAM Keys are compared with ==. Because 1 == 1.0 == True and they have the same hash, they are the same key: the dict keeps the first key object and the last value. Use one type for your keys.

STEP 4 Core Python · Functions

4.1 Functions: def, return and every kind of argument

def and return def name(params): body. return value leaves the function. No return gives None. print is not return.

Argument kinds Positional by order, keyword by name, positional before keyword. Defaults make parameters optional.

Star parameters *args tuple of extra positionals, **kwargs dict of extra keywords. Bare * means keyword-only after it, / means positional-only before it.

Mutable default Defaults are made once at def time. Use None, then create the list inside.

Functions are objects Assign them, store them, pass them: sorted(key=f), map(f, items). Docstring on the first line, type hints are not enforced.

A function maps each input to one output
$f: X \to Y,\ x \mapsto f(x)$

Composition $(f \circ g)(x) = f(g(x))$

EXAM A PARAMETER is the name in the def line (name, width). An ARGUMENT is the value you pass in the call ("Asha", 4). Exams use both words, so learn the difference.

EXAM A function with no return, or with a bare return, gives back NONE. That is why a = show_double(5) printed 10 but left a as None.

EXAM describe(name="Sara", 25) is a SyntaxError: a positional argument may not follow a keyword argument. Python refuses to even start the program.

EXAM In a def line, * and ** COLLECT arguments. In a call, * and ** SPREAD a list or dict out into separate arguments. Same symbols, opposite jobs.

4.2 Scope and recursion: LEGB, closures and the call stack

Scopes Local (inside a call), Enclosing, Global (module), Built-in: LEGB. Assigning makes a name local in the whole function.

global and nonlocal global x assigns the module x. nonlocal x assigns the enclosing function x. Reading needs neither.

Closures An inner function that remembers enclosing names (cells). Used for counters, factories and decorators.

Recursion Base case + smaller recursive case. Each call is a frame. Limit about 1000, RecursionError.

Costs T(n) = T(n-1) + c is O(n). Naive fib 2F(n+1) - 1 calls, lru_cache makes it n + 1. Hanoi 2^n - 1. Fast power O(log n).

Recurrence for a simple recursion
$T(n) = T(n-1) + c$ gives $T(n) = O(n)$

Calls made by naive fib(n)
$C(n) = C(n-1) + C(n-2) + 1 = 2F(n+1) - 1$

Tower of Hanoi moves
$M(n) = 2M(n-1) + 1 = 2^n - 1$

EXAM Assigning to a name inside a function makes a new local name, even if a global with the same name exists. This is called SHADOWING. Changing an object (items.append) is different from rebinding a name (items = [...]): the first is seen by the caller, the second is not.

EXAM Python decides which names are local when it compiles the function, not while it runs. If a name is assigned anywhere in the function, it is local in the whole function.

EXAM global reaches the module scope. nonlocal reaches the nearest enclosing function scope and is a SyntaxError at the top level. Decorators (Advanced 1.2) are built from closures.

TRAP Each call uses a frame on the call stack. To protect the program, CPython stops at about 1,000 frames deep with RecursionError. sys.setrecursionlimit can raise the limit, but a recursion that deep is usually better written as a loop.

4.3 lambda, map, filter and comprehensions

lambda lambda args: expression. One expression, no statements. Best as an argument, def otherwise.

key functions sorted/min/max(items, key=f). Tuple keys sort by several fields. -x reverses a number key.

map, filter, reduce Lazy iterators: list(map(f, a)), list(filter(test, a)). functools.reduce(f, a, start) folds to one value.

Comprehensions [expr for x in a if test], [a if t else b for x in s], {k: v for ...}, {x for ...}. Outer for first.

Generator expressions (expr for x in a): lazy, one use, tiny memory. sum(x*x for x in a), any(...), all(...).

List comprehension
[expression for item in iterable if condition]

With if-else
[a if test else b for item in iterable]

Set builder notation $\{ f(x) \mid x \in S, p(x) \}$

reduce is a fold
$\text{reduce}(\oplus, [a_1, a_2, a_3], s) = ((s \oplus a_1) \oplus a_2) \oplus a_3$

EXAM A lambda cannot contain statements such as if blocks, for loops, return or assignment with =. It can contain a conditional expression: lambda n: "even" if n % 2 == 0 else "odd".

TRAP map and filter return iterators, not lists. They give each value once, only when asked. Wrap them in list() to see all the values, and do not loop over the same map object twice.

EXAM [n for n in nums if n % 2 == 0 else 0] is a SyntaxError. An if after for only filters and has no else.

STEP 5

Core Python · Errors, modules and files

5.1 Exceptions: try, except, else, finally and raise

try and except try: risky code. except X as err: handles X and its children. First matching handler only.

else and finally else runs if no exception. finally always runs (cleanup). Order: try, except..., else, finally.

Hierarchy BaseException > Exception > ArithmeticError > ZeroDivisionError. LookupError > KeyError, IndexError. KeyboardInterrupt is not an Exception.

raise raise ValueError("msg"). Bare raise re-raises. raise New() from err sets __cause__. Custom: class MyError(Exception).

Style Catch specific classes, never bare except. assert is for programmer checks (-O removes it). EAFP. with closes resources.

Handlers checked before a match
position of the first matching except clause

Chance that at least one of n independent steps fails
$1 - (1 - p)^n$

TRAP Avoid a bare except: or except BaseException:. They also catch KeyboardInterrupt and SystemExit, so Ctrl+C stops working. Catch Exception at most, and only where you log or report the error.

EXAM The order is fixed: try, then except clauses, then else, then finally. You need at least one except or a finally. else keeps code out of the try block, so you do not accidentally catch errors from it.

5.2 Modules and packages: import, the standard library and pip

import forms import m, import m as a, from m import x, from m import x as y. Avoid import *. Relative: from . import x.

How import works Check sys.modules, search sys.path, run the file once, cache it, bind the name. importlib.reload re-runs.

math and random ceil, floor, trunc, sqrt, isqrt, hypot, factorial, comb, gcd, log, isclose. randint(a, b) includes b, randrange excludes stop, seed repeats.

Packages Folder + __init__.py. _name is private by convention. __all__ controls import *. __pycache__ holds .pyc files.

Main guard and pip if __name__ == "__main__": main(). pip install inside a venv. pip freeze > requirements.txt.

Combinations and permutations
$C(n, k) = n! / (k!(n-k)!),\ P(n, k) = n! / (n-k)!$

Euclid: greatest common divisor
$\gcd(a, b) = \gcd(b, a \bmod b),\ \gcd(a, 0) = a$

EXAM floor goes down and trunc goes toward zero. They agree for positive numbers and differ for negative ones: floor(−4.8) is −5, trunc(−4.8) is −4. round() is a built-in, not in math, and rounds halves to the even number.

TRAP random gives PSEUDO-RANDOM numbers from the Mersenne Twister algorithm: good for games and simulations, but predictable. For passwords, tokens and anything secret use the secrets module (secrets.token_hex(16)).

TRAP Any file name.py that Python can find is a module called name. Use lower case names with underscores, and never name your file after a standard module: a file called random.py in your folder hides the real random module, and import random gives you your own file.

5.3 Files: reading, writing, modes, paths and streams

open and with with open(path, mode, encoding="utf-8") as f: closes automatically. Modes r, w (empties), a, x (new only), +, b.

Reading read(), read(n), readline() ("" at end), readlines(), for line in f (best for big files). Lines keep their new line.

Writing write(s) and writelines(list) add no new lines. print(x, file=f) does. write returns characters written.

Binary and cursor "rb"/"wb" use bytes. bytearray + readinto. seek(n) moves, tell() reports.

Errors and paths OSError family: FileNotFoundError (ENOENT), FileExistsError, PermissionError. pathlib.Path / "x", glob, read_text.

File size in bytes of an ASCII text
sum of len(line) for all lines, new lines included

UTF-8 size
1 byte for code points 0 to 127, 2 up to 2,047, 3 up to 65,535, 4 above

EXAM Lines keep their new line character. Use line.strip() or line.rstrip("\n") before you compare or split them. For big files loop with for line in f, which reads one line at a time instead of loading gigabytes into memory.

TRAP Add t (text, the default) or b (binary) to any mode: "rb", "wb", "ab", "r+b". The most common accident is opening a file with "w" when you meant "a": the old contents are gone at once.

TRAP Without encoding=..., open() uses a default that depends on the computer. It is UTF-8 on Linux and macOS but can be an old code page on Windows, so a file written on one computer may not read on another. Since Python 3.15 the default becomes UTF-8 everywhere (PEP 686), but writing encoding="utf-8" is still the safe habit.

TRAP os.remove, os.rename and shutil.rmtree do not use a recycle bin: deleted files are gone. Print the paths and check them before you delete anything in a loop.

TRAP Never split CSV lines with line.split(","). A value like "Ravi, Jr" contains a comma and is stored in quotes. The csv module handles quotes, commas and new lines inside values correctly. Advanced 6.2 covers csv, json, xml and sqlite3 in depth.

STEP 6

Core Python · Object oriented programming

6.1 Classes and objects: attributes, methods, self and encapsulation

Class and object class C: defines a blueprint. C(...) makes an object and runs __init__(self, ...). self is the object.

Two kinds of attribute self.x is per object (in obj.__dict__). x = ... in the class body is shared. Read looks in the object, then the class. Write goes to the object.

Shared list trap A list or dict class attribute is shared by all objects. Create it in __init__.

Text forms __str__ for print and str, __repr__ for repr, lists and debugging. Write __repr__ at least.

Introspection and privacy type, isinstance, hasattr, getattr, setattr, vars, __name__, __module__, __bases__. _x is a convention, __x becomes _Class__x. is = same object.

Length of a vector (Pythagoras)
$|v| = \sqrt{(x^2 + y^2)}$

Dot product $a \cdot b = a_x b_x + a_y b_y$

EXAM self is not a keyword. It is only the name every Python programmer uses for the first parameter of a method. Python fills it in for you: rex.bark() becomes Dog.bark(rex). Forget self in the def line and the call fails with "takes 0 positional arguments but 1 was given".

EXAM Writing obj.attr = value never goes to the class. It always creates or changes the attribute on that one object. That is why a.species = "wolf" left b.species and Dog.species alone.

TRAP A class attribute that holds a list, dict or set is ONE object for all instances. Changing it with append or update through any object changes it for every object. Create such data in __init__ with self.items = [] instead. This is the same trap as a mutable default argument (4.1).

EXAM Inside a method, change a class attribute through the class name: Student.count += 1. Writing self.count += 1 reads the class value but then stores the result on the object, so the class counter would never change.

EXAM ENCAPSULATION means an object guards its own data and offers methods to work with it, so outside code cannot put it into a bad state (like a negative balance). Python trusts programmers: it has no real private keyword like Java. Use one underscore for internal things, and properties (Advanced 1.3) when you need checks on reading or writing.

EXAM a is b asks "are these the same object". a == b asks "are these equal". For your own classes == means the same as is until you write __eq__. Writing __eq__ sets __hash__ to None, so the objects can no longer be set items or dict keys unless you also write __hash__ (Advanced 1.1).

6.2 Inheritance: super(), overriding, polymorphism and the MRO

Inherit and override class Child(Parent): reuses Parent. Writing a method again overrides it. Lookup: object, class, then parents in MRO order, ending at object.

super() super().__init__(...) runs parent setup. super().m() extends a method. super() means the next class in the MRO of the object.

Checks isinstance counts children, type(x) is C does not. issubclass(bool, int) is True. __bases__ and __mro__ show the family.

Polymorphism One call works on many types. Duck typing needs only the method. Abstract methods: NotImplementedError or abc with @abstractmethod.

MRO and composition C3: child before parents, parents in written order, each once. Impossible order is TypeError. Prefer has a over is a.

C3 linearisation
$L[C] = C + \text{merge}(L[B_1], ..., L[B_n], [B_1, ..., B_n])$

Circle $A = \pi r^2$

Heron for a triangle
$s = (a + b + c) / 2$, $A = \sqrt{(s(s - a)(s - b)(s - c))}$

EXAM An overriding method normally keeps the same parameters as the parent one, so code that works with an Animal also works with a Dog. This rule is called the Liskov substitution principle.

TRAP A child __init__ replaces the parent one completely. If it does not call super().__init__(...), the attributes the parent sets are never created.

EXAM Prefer isinstance over type(x) == C, because a child object is also a valid parent object. Every class inherits from object in the end, and bool is a child of int, which is why True + True is 2.

EXAM super() does not mean "my parent". It means "the next class in the MRO of the object I am working on". Inside B, super().hello() went to C for a D object but to A for a B object. That is what lets every class in a diamond run exactly once.

STEP 7

Core Python · Exam practice

7.1 Exam practice: PCEP and PCAP style papers

Exam facts PCEP-30-02: 30 questions, 40 min, 70 %. PCAP-31-03: 40 questions, 65 min, 70 %. OOP is 34 % of PCAP.

Operator traps -2 ** 2 is -4, 2 ** 3 ** 2 is 512, -7 // 2 is -4, -7 % 3 is 2, round(2.5) is 2, 0.1 + 0.2 != 0.3.

Collection traps b = a aliases, [[0] * 3] * 3 shares a row, (1,) is a tuple, {} is a dict, sort returns None.

Function traps Mutable defaults are shared, assignment makes a name local, lambdas in loops bind late, finally always runs.

Object traps obj.x = v never changes the class, __x becomes _Class__x, a child __init__ must call super().__init__().

Time per question
minutes × 60 / questions

Questions to pass (if every item counts the same)
correct needed = ceil(pass mark × questions)

TRAP The Python Institute has announced newer versions, PCEP-30-03 (35 questions, 50 minutes, 70 %) and PCAP-31-04 (42 questions, 65 minutes, 72 %), released on 31 October 2026. The skills tested are the same core Python, but check which version your voucher is for before you book.

EXAM In the exam: read the whole snippet before the options, trace it line by line on scratch paper, watch for indentation and for a missing comma or bracket that turns the answer into "the code raises an error". Mark hard questions and come back, every question is worth the same.

STEP 1

Advanced Python · Advanced object oriented programming

1.1 Special methods and operator overloading

How operators work a + b calls type(a).__add__(a, b). Missing or NotImplemented: try type(b).__radd__(b, a), then TypeError. Lookup is on the class.

Text __repr__ (developers, lists, REPL), __str__ (print), __format__ (f-string specs).

Compare and hash __eq__ plus __lt__ with total_ordering. Equal objects need equal hashes. __eq__ alone makes a class unhashable.

Containers __len__, __getitem__ (ints and slices), __setitem__, __contains__, __iter__. __bool__ falls back to __len__.

Other hooks __call__ for callable objects, __iadd__ returns self, __getattr__ only on failed lookup, __setattr__ on every assignment.

Adding fractions $a/b + c/d = (ad + cb) / bd$

Lowest terms
divide top and bottom by g = gcd(top, bottom)

EXAM Special methods are looked up on the CLASS , not on the object. Setting obj.__len__ = ... on one object does nothing for len(obj). Also, never invent your own __names__: they are reserved for Python.

EXAM Return the special value NotImplemented (not raise NotImplementedError) when your method cannot handle the other type. That gives the other object a chance to answer. Raising stops Python from trying.

EXAM x += y first tries x.__iadd__(y) and then ASSIGNS the result to x. That is why __iadd__ must return self. A tuple has no __iadd__, so += makes a new tuple and other names still see the old one.

1.2 Extended arguments, closures and decorators

Parameters a, / positional only. *args tuple. after * keyword only. **kwargs dict. f(*seq, **d) unpacks at a call.

Closures Functions are objects. An inner function remembers the variables of the outer one (__closure__), even after it returned.

Decorator shape @d def f is f = d(f). def d(func): @wraps(func) def wrapper(*args, **kwargs): ... return func(*args, **kwargs). return wrapper.

Arguments and stacking @d(x) is f = d(x)(f): three levels. @a @b def f is a(b(f)): applied bottom up, run top down.

Built ins @cache, @lru_cache(maxsize), @wraps, @property, @classmethod, @staticmethod, @dataclass, @contextmanager. Cache only pure functions.

Calls of plain recursive Fibonacci
$C(n) = 1 + C(n-1) + C(n-2)$, $C(0) = C(1) = 1$, so $C(n) = 2F(n+1) - 1$

Calls with @cache
each n from 0 to N is computed once: N + 1 misses

EXAM The order in a def is always: positional only, /, normal, *args (or a bare *), keyword only, **kwargs. *args, **kwargs in a wrapper accept any call and pass it on unchanged, which is why every general decorator uses them. Notice the fourth line of output: name="Asha" did not fill name. Because describe has **extra, it went into extra, so Python reports name as missing.

TRAP Without @wraps every decorated function is called "wrapper" and loses its docstring, which breaks help(), debuggers, logging and frameworks that look up functions by name (Flask uses the name for its routes). Always put @wraps(func) on the wrapper.

TRAP A decorator with arguments has three nested functions: the factory (settings), the decorator (the function) and the wrapper (the call). Forgetting the brackets, @retry instead of @retry(3), passes the function as attempts and fails in a confusing way.

EXAM Decorators are applied from the bottom up (nearest the def first) and run from the top down at call time. @a then @b above def f means f = a(b(f)).

TRAP Only cache PURE functions, whose result depends only on the arguments. Arguments must be hashable. @cache never forgets, so for many different arguments use lru_cache with a maxsize.

1.3 Class design: methods, properties, abstract classes and dataclasses

Method kinds Instance method gets self. @classmethod gets cls (alternative constructors, returns cls(...)). @staticmethod gets nothing.

Properties @property getter, @x.setter with checks, no setter means read only. Store the value as self._x. @cached_property computes once.

Abstract classes class C(ABC) with @abstractmethod: C() and incomplete children raise TypeError. collections.abc offers ready contracts.

Dataclasses @dataclass writes __init__, __repr__, __eq__. field(default_factory=list), frozen, order, __post_init__, kw_only, slots, asdict, replace.

Slots and descriptors __slots__ removes __dict__. Descriptors have __get__ and __set__. Data descriptors beat the instance dict.

Celsius to Fahrenheit (a computed property)
$F = C \times 9 / 5 + 32$

order=True compares as tuples
$(a_1, a_2, ...) < (b_1, b_2, ...)$ at the first position where they differ

EXAM Student.from_text returned a Student because the method calls cls(...), not Person(...). That is why alternative constructors should be class methods.

TRAP Store the real value under another name (self._celsius). If the setter wrote self.celsius = value it would call itself forever and raise RecursionError.

EXAM A DESCRIPTOR is an object with __get__, __set__ or __delete__, stored as a class attribute. property, classmethod, staticmethod and every normal method are descriptors. Write your own when the same rule (like "must be positive") applies to many attributes in many classes.

1.4 Exceptions in depth, copying and serialisation

Exception design class AppError(Exception) plus children. super().__init__(message). Catch the child for one problem, the base for all.

Chaining raise New() from err sets __cause__. Raising inside except sets __context__. from None hides it. err.__traceback__ has the frames.

3.11 tools err.add_note(text) and err.__notes__. ExceptionGroup(msg, [errors]) caught with except* Type.

Copies b = a no copy. a.copy(), a[:], list(a), dict(a), copy.copy: shallow. copy.deepcopy: everything, loops handled by memo.

Saving objects pickle.dumps/loads (bytes), dump/load (files in "wb"/"rb"). Never unpickle untrusted data. shelve: dict on disk, returns copies.

Chance that k independent attempts all fail
$P(\text{all fail}) = p^k$, $P(\text{success}) = 1 - p^k$

Objects created by copies of an n by m nested list
shallow: 1 list. deep: 1 + n lists

EXAM Use raise ... from err when you turn a low level error into a meaningful one, so the original is never lost. Use from None only when the original adds nothing for the reader.

TRAP Never unpickle data from a source you do not trust. pickle.loads can run any code the data asks for, so a crafted file can take over your computer. For data from users or the internet use JSON.

EXAM A shelf returns copies. db["ravi"]["tags"].append("work") changes a copy that is thrown away. Read the value, change it and assign it back (or open with writeback=True).

1.5 Metaprogramming: type, class hooks and metaclasses

Classes are objects type(x) gives the class. type(Dog) is the metaclass (normally type). type(type) is type. Classes can be stored, passed and returned.

type() with three arguments type(name, bases, namespace) builds a class. A class statement does the same. The body runs once and fills the namespace.

Introspection getattr(obj, name, default), setattr, hasattr, delattr, callable, vars, dir, __class__, __bases__, __mro__, __subclasses__(), inspect.signature.

Hooks __init_subclass__ on the parent runs for every child (registries, checks). Class decorators get the finished class.

Metaclasses class Meta(type): __new__ builds the class, __call__ makes objects. Inherited by children. Conflicts raise TypeError. Prefer lighter tools.

Classes in a full hierarchy below one base
$b + b^2 + ... + b^d = b(b^d - 1) / (b - 1)$

Average checks in an if and elif chain
average = (n + 1) / 2 for n equally likely names

EXAM type has two jobs. With one argument, type(x) tells you the class of x. With three arguments, type(name, bases, namespace) builds a new class. And type(type) is type: type is its own class, which is where the chain ends.

TRAP The class body runs once, at the moment Python reaches the class statement, not each time you make an object. Names it creates (tax, price, total) end up in the namespace dict, which becomes the class __dict__. That is why a class body can hold any normal code, even print.

EXAM __init_subclass__ (PEP 487, Python 3.6) runs on the PARENT every time a child class is created. It is a class method automatically. Keyword arguments in the class line, like name="csv", are passed to it. Always call super().__init_subclass__(**kwargs) so other parents in a multiple inheritance chain get their turn.

EXAM A metaclass is a subclass of type. Its __new__(mcs, name, bases, namespace) gets everything the class statement collected and must return the new class, normally from super().__new__. Children of Job use CheckMeta too, because Python uses the metaclass of the base class when none is given. The check fires when the class is DEFINED, long before anyone uses it.

TRAP Calling a class, Config(...), really calls type(Config).__call__. The normal type.__call__ runs __new__ and then __init__. Overriding __call__ in a metaclass changes what calling the class does: here it returns the saved object. Notice that the second call did not even run __init__, so mode stayed "prod".

STEP 2

Advanced Python · Iterators, generators and context managers

2.1 Iterators and generators

Iterator protocol iter(x) gives an iterator. next(it) gives a value or raises StopIteration. next(it, default) avoids the error. A for loop does this for you.

Iterable versus iterator Iterables (list, range, dict) give a fresh iterator each time. Iterators (zip, map, files, generators) are used up after one pass.

Generators A function with yield returns a generator. Nothing runs until next(). It pauses at each yield and keeps its locals. Return ends it.

Pipelines and laziness Chain generators: values are made only when asked, memory stays constant, and islice or break stops work early.

Advanced control yield from delegates and returns the inner return value. send(v), throw(e), close() (GeneratorExit). Prime with next() before send.

Memory of a list against a generator
list: about 8n bytes of pointers plus the items, generator: a constant (about 200 bytes)

Work done by a lazy search
items read = position of the first match (k), not n

EXAM iter(nums) gave a NEW iterator each time, so the first answer was False. An iterator returns itself from iter(), so the second was True. That is the whole difference: an iterable can make many iterators, an iterator is one walk that can be used once.

TRAP Raising StopIteration ends the loop. Do not raise it inside a generator body: since Python 3.7 (PEP 479) it is turned into RuntimeError. In a generator, simply return.

EXAM A function that contains yield anywhere is a generator function. Calling it runs nothing: the first line, print("start"), only ran at the first next(). After the last value the generator is closed, and list() on a used generator gives an empty list.

TRAP A common bug: saving map(...) or zip(...) in a variable, looping over it once for a check, and then finding it empty the second time. Wrap it in list(...) if you need it twice.

2.2 itertools, functools and operator

Endless and slicing count, cycle, repeat never stop: use islice, zip, takewhile. takewhile/dropwhile cut at the first failure. compress applies a mask.

Joining chain, chain.from_iterable, zip_longest(fillvalue=), pairwise, batched, starmap, accumulate(func, initial=), tee.

groupby Joins neighbours with equal keys only. Sort by the same key first. Use each group before moving on.

Combinatorics product n**r, permutations n!/(n-r)!, combinations C(n,r), with replacement C(n+r-1,r). math.comb and math.perm count without listing.

functools and operator reduce (left fold), partial, singledispatch, lru_cache and cache_info. itemgetter, attrgetter, methodcaller as key functions.

product(items, repeat=r) n^r

permutations(items, r) $P(n, r) = n! / (n - r)!$

combinations(items, r)
$C(n, r) = n! / (r! (n - r)!)$

Prefix sums with accumulate
P = accumulate(a, initial=0), sum of a[i] to a[j] = P[j + 1] − P[i]

TRAP count, cycle and repeat never end on their own. Always stop them: with islice, zip with something finite, takewhile, or a break.

TRAP groupby only joins items that are NEXT to each other. On unsorted data tea appeared twice. Sort by the same key first, or use a dict (collections.defaultdict(list)) when you cannot sort.

EXAM singledispatch chose the int version for 21, the list version for [1, 2] and the base version for 2.5. It looks at the type of the FIRST argument only. For methods use functools.singledispatchmethod.

2.3 Context managers and the with statement

The with protocol with X as v: v = X.__enter__(), body, then X.__exit__(type, err, tb) always. Methods are looked up on the type.

Suppressing __exit__ returning True hides the exception. contextlib.suppress(Error) hides only the listed types.

@contextmanager Set up, then yield the as value inside try, then clean up in finally. Re raise errors you do not handle. Works as a decorator.

Standard managers open, Lock, TemporaryDirectory, chdir, redirect_stdout, localcontext, closing, nullcontext, sqlite3 (transaction), mock.patch.

Several managers Entered left to right, exited right to left. Bracketed form across lines. ExitStack: enter_context and callback for a dynamic number.

Exit order of n nested managers
the k-th one entered is the (n − k + 1)-th one exited

Chance that at least one of n steps fails
$1 - (1 - p)^n$

TRAP Only return True from __exit__ on purpose and only for the exact errors you expect. Hiding every error makes bugs invisible. contextlib.suppress(SomeError) is the clear, ready made way to say "this error is fine here".

EXAM Everything before yield is the set up, the yielded value goes into as, and everything after yield is the clean up. The try and finally around the yield is what makes the clean up run after an error: without it, an exception in the body would be raised at the yield and skip the rest.

EXAM Managers are entered left to right and exited in the reverse order, like a stack: the last one opened is the first one closed. That is what you want: an inner resource that depends on an outer one is cleaned up first.

STEP 3

Advanced Python · Professional code: style, testing and logging

3.1 Clean code: PEP 8, docstrings and type hints

PEP 8 layout 4 spaces, 79 characters, 2 blank lines around top level defs, imports one per line grouped stdlib, third party, local.

PEP 8 names snake_case functions and variables, CapWords classes, UPPER_CASE constants, _private by convention.

Bug preventing habits is None, if items:, isinstance, startswith and endswith, None instead of mutable defaults, small try blocks.

Docstrings PEP 257: """One line summary.""" then details. __doc__ and help() read it. doctest runs >>> examples.

Type hints and tools list[int], int | None, Callable, Protocol, TypedDict, Literal, Final, def f[T]. Not checked at run time: use mypy, ruff and black.

Cyclomatic complexity (McCabe)
$M = E - N + 2$ (edges and nodes of the control flow graph) = decision points + 1

Paths through k independent if statements
paths = 2^k

EXAM black fixed the layout but not the names, the unused import or == None: a formatter only moves spaces and line breaks, it never changes meaning. That is why teams use a formatter AND a linter. ruff can do both (ruff format and ruff check).

EXAM Hints are stored in __annotations__ but Python never checks them while running: first("hello") worked although the hint says list. In Python 3.14 annotations are evaluated lazily (PEP 649 and 749), so a hint can even name a class defined later in the file.

3.2 Testing: unittest, pytest and mocks

unittest class T(unittest.TestCase), methods test_..., assertEqual, assertTrue, assertIn, assertRaises (with), assertAlmostEqual, setUp, subTest, skip.

Outcomes PASSED, FAILED (assert false), ERROR (other exception). Read the last line. Tests must not depend on each other.

Mocks Mock().m.return_value, side_effect (list or exception), assert_called_once_with, call_count, patch and patch.object. Inject dependencies.

pytest test_ functions with plain assert, fixtures by parameter name, parametrize, approx, raises(match=), tmp_path, monkeypatch, -v, -k.

Strategy Edges and just outside, every fixed bug, coverage is not correctness, property tests with Hypothesis, the test pyramid, TDD.

Line coverage
coverage = lines run / total lines × 100 %

Boundary value tests for a range a to b
test a − 1, a, b, b + 1 (and one value inside)

Binary search steps (the LeetCode set)
steps ≤ $\lceil \log_2(n + 1) \rceil$

EXAM unittest is built into Python. A test is a method whose name starts with test_ inside a class that inherits from unittest.TestCase. assertEqual, assertTrue, assertIn, assertRaises and assertAlmostEqual (for floats) check results and give a clear message when they fail.

TRAP Tests must not depend on each other or on their order. test_add_more added a book, but test_total still saw only the pen, because setUp made a new cart for each test.

TRAP 100 % coverage means every line ran during the tests, NOT that every case was checked. The next lesson shows a bug that survives full coverage.

EXAM All 8 example tests passed with 100 % coverage, yet price − price × 100 / 100 can come out a tiny bit below zero because of float rounding. The fix is price × (100 − percent) / 100, which gives exactly 0 at 100 %. The failing price changes from run to run, because Hypothesis tries random inputs.

3.3 Logging and debugging

Levels DEBUG 10, INFO 20, WARNING 30 (default), ERROR 40, CRITICAL 50. basicConfig(level=, format=, stream=, force=True).

Parts getLogger(__name__), logger and handler levels, Formatter fields (%(asctime)s %(levelname)s %(name)s %(message)s), propagate to parents.

Habits logger.info("x %s", x) not f-strings, logger.exception in except, no secrets in logs, libraries never configure logging.

Configuration dictConfig once at start, RotatingFileHandler (maxBytes, backupCount), TimedRotatingFileHandler, QueueHandler.

Debugging f"{x=}", assert (removed by -O, not for input checks), breakpoint() and python -m pdb: b, c, n, s, p, l, w, q.

Log volume
bytes = lines per second × bytes per line × seconds

Disk used by a rotating log
max size = (backupCount + 1) × maxBytes

EXAM Without basicConfig the root logger shows only WARNING and above, on stderr. basicConfig sets the level, the format and the stream in one call, but only the first time, unless you pass force=True.

EXAM Pass values as arguments: logger.debug("total %s", total). The text is only built if the record passes the level checks. An f-string is always built, even when the message is thrown away, as the first slow __str__ line shows.

TRAP Inside except, logger.exception("...") logs at ERROR level and adds the full traceback. Never log passwords, card numbers or personal data: logs are read by many people and kept for a long time.

TRAP assert checks a condition you believe is always true and raises AssertionError if not. Python started with -O skips every assert, so never use assert to check user input. Raise ValueError for that.

STEP 4

Advanced Python · Speed: complexity and smart data structures

4.1 Complexity, timing and profiling

Big O Drop constants, keep the biggest term. Loops in a row add, nested loops multiply. O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2^n).

Costs list: index O(1), append O(1) amortized, x in list O(n), insert(0)/pop(0) O(n). dict and set: O(1) average. sort O(n log n).

Measure timeit (python -m timeit -s setup stmt, best of several runs), time.perf_counter, realistic n.

Profile python -m cProfile -s cumulative file.py, tottime vs cumtime, fix only the hot spot. tracemalloc for memory, getsizeof is shallow.

Trade offs A better algorithm first. Sets for lookups, lru_cache for repeated work, generators to save memory.

Definition of Big O
f(n) = O(g(n)) if there are numbers c > 0 and n_0 with f(n) ≤ c · g(n) for every n ≥ n_0

Sum of a loop inside a loop
1 + 2 + ... + n = n(n + 1) / 2 = O(n²)

Geometric sum (doubling)
1 + 2 + 4 + ... + n = 2n − 1 < 2n

Master theorem
T(n) = a·T(n/b) + O(n^d) gives O(n^d) if d > $\log_b$ a, O(n^d log n) if d = $\log_b$ a, O($n^{\log_b a}$) if d < $\log_b$ a

TRAP Hidden loops count too. x in my_list, my_list.index(x), my_list.remove(x), my_list.insert(0, x), "".join and slicing a[i:j] each walk through items. Putting one of them inside a loop is the most common way Python code becomes quadratic by accident.

EXAM Average case versus worst case: a dict lookup is O(1) on average, but if many keys had the same hash it could take O(n). Python adds random salt to str hashes so attackers cannot cause this on purpose. For interviews and exams, dict and set operations are O(1).

EXAM sys.getsizeof is shallow. For a list it counts the list and its slots, not the objects inside, which is why a list holding a huge number has the same size as one holding a small number. tracemalloc counts every allocation made while it runs, so it shows the real memory a piece of code needed.

4.2 collections, heapq and bisect

Counter Counter(items), c[x] is 0 if missing, most_common(k), total(), elements(), + - & | on counts.

deque append, appendleft, pop, popleft all O(1), rotate(n), deque(maxlen=n). Queues and BFS use popleft.

Records namedtuple("P", ["x", "y"]) with _replace, _asdict, _fields. OrderedDict.move_to_end, popitem(last=False). ChainMap(first, fallback). defaultdict(list).

heapq heappush, heappop O(log n), heapify O(n), heap[0] smallest, nlargest and nsmallest O(n log k), merge, (priority, count, item), heappush_max in 3.14.

bisect bisect_left, bisect_right, insort, key= since 3.10. Range count = right(high) − left(low). Only on sorted lists.

Heap indexes
parent(i) = (i − 1) // 2, children 2i + 1 and 2i + 2

Height of a heap h = $\lfloor \log_2 n \rfloor$

Top k with a heap
O(n log k) instead of O(n log n) for a full sort

EXAM A deque (say "deck") is a double ended queue. Adding and removing at either end is O(1). A list does the same at the end, but list.insert(0, x) and list.pop(0) must shift every other item, which is O(n).

EXAM Since Python 3.7 every dict keeps insertion order. OrderedDict still has extras: move_to_end(key), popitem(last=False) for the oldest item, and == compares order too. That makes it the classic tool for an LRU cache.

EXAM A heap is a list where every item is smaller than or equal to its two children: heap[i] ≤ heap[2i + 1] and heap[i] ≤ heap[2i + 2]. So heap[0] is always the smallest. The rest is NOT sorted. heappush and heappop are O(log n), heapify is O(n), and looking at heap[0] is O(1).

STEP 5

Advanced Python · Doing many things at once: threads, processes and asyncio

5.1 Threads and processes

When to use what I/O bound: threads or asyncio. CPU bound: processes (or python3.14t). Measure first.

Threads Thread(target, args), start, join, daemon. Shared memory means races: protect with with lock:.

Sync tools Lock, RLock, Semaphore(n), Event, Condition, Barrier, queue.Queue. Same lock order everywhere to avoid deadlock.
Executors ThreadPoolExecutor and ProcessPoolExecutor: submit, map (keeps order), as_completed (finish order), result() raises errors.
Processes if __name__ == "__main__" guard, picklable functions and data, start methods fork, spawn, forkserver (Linux default in 3.14).

Speedup and efficiency $S = T_1 / T_n$, $E = S / n$

Amdahl's law $S(n) = 1 / ((1 - p) + p / n)$

Threads for I/O work
threads ≈ cores × (1 + wait time / compute time)

TRAP Every thread in a process shares the same global variables and objects. That makes it easy to pass data, and also easy to break data: two threads changing the same object at the same time is a race condition.

TRAP counter = counter + 1 is really three steps: read, add, write. If another thread writes between your read and your write, your write wipes out its update. time.sleep(0) here forces a switch at the worst moment, but real programs hit the same bug by chance, often only under heavy load.

EXAM A DEADLOCK happens when thread 1 holds lock A and waits for lock B while thread 2 holds B and waits for A. Neither can go on. Prevent it by always taking locks in the same order, holding them briefly, or using a timeout.

EXAM With the GIL, 4 threads are no faster than one (1.37 s against 1.30 s). 4 processes are 3.3 times faster. The free-threaded build python3.14t (officially supported since 3.14) has no GIL, so there 4 threads are fast too.

5.2 asyncio: async and await

Basics async def, await (only inside async def), asyncio.run(main()) once. Calling a coroutine function does not run it.
Together gather (results in order), TaskGroup with except*, create_task (keep a reference), as_completed (finish order).
Control asyncio.timeout(s) and wait_for, task.cancel() and CancelledError (raise it again), Semaphore(k) to limit.
Pipelines asyncio.Queue(maxsize) with await put and get for back pressure. Async generators, async for, async with.
Never block No time.sleep or requests in coroutines. Use asyncio.sleep, httpx.AsyncClient, asyncio.to_thread or run_in_executor.

Time one by one $T = n \times t$

Time with all at once $T \approx \max(t_i) + \text{overhead}$

With a limit of k at once $T \approx \lceil n / k \rceil \times t$

TRAP Calling make_tea(...) only creates a coroutine object, it runs nothing. The code runs when it is awaited, or when it is wrapped in a task (gather, create_task, TaskGroup). Forgetting await is the most common asyncio bug: Python warns "coroutine ... was never awaited".

TRAP Keep a reference to every task you create with create_task (in a list or a set), or use a TaskGroup. The loop holds tasks only weakly, so a task nobody refers to can disappear before it finishes.

EXAM Cancelling a task raises CancelledError inside it at its current await. Clean up in finally blocks, and do not swallow CancelledError without raising it again, or the task cannot be stopped.

TRAP A normal blocking call (time.sleep, requests.get, a slow calculation) inside a coroutine freezes the whole loop. Use the async version (asyncio.sleep, httpx.AsyncClient, aiofiles), or push the blocking call to a thread with asyncio.to_thread, or to a process pool with loop.run_in_executor.

STEP 6

Advanced Python · The standard library toolbox: text, data, databases and GUIs

6.1 Regular expressions

Syntax . \d \w \s [abc] [^abc] ^ $ \b, repeats * + ? {m,n}, lazy *? +?, | and ( ). Always raw strings.
Functions search (anywhere), match (start), fullmatch (whole), findall (strings), finditer (match objects), sub, subn, split.
Groups m.group(n), m.groups(), (?P<name>...), m.groupdict(), (?:...) non-capturing, \1 backreference, \g<name> in sub.
Flags IGNORECASE, MULTILINE, DOTALL, VERBOSE. re.compile for named patterns, re.escape for literal text.
Care Greedy versus lazy, lookarounds (?=) (?!) (?<=) (?

Choices for a character class
patterns of length n over a class of size k match k^n strings

A regular language
built from single characters with union (|), concatenation and star (*)

Catastrophic backtracking
ways to split n letters among nested repeats grows like 2^n

EXAM Use fullmatch to check input such as a postcode or a username. search would also accept "xx12345yy" because it finds 12345 inside it.

TRAP findall changes what it returns when the pattern has groups: with one group it returns that group's text, with several it returns tuples. Use (?:...) for groups you only need for | or repeats.

TRAP Some patterns make the engine try a huge number of ways before failing, for example (a+)+$ on "aaaaaaaaaaaaaaaaaaaaaaaaaaaa!". This is CATASTROPHIC BACKTRACKING , and attackers use it to freeze servers (ReDoS). Avoid nested repeats of the same thing, or use possessive quantifiers.

6.2 CSV, JSON, XML and settings files

CSV csv.DictReader, DictWriter(f, fieldnames), writeheader, open(..., newline=""), delimiter, quoting. Values are strings.
JSON dumps and loads, dump and load, indent, sort_keys, ensure_ascii, default=, object_hook, JSONDecodeError. Tuples become lists, keys strings.
XML ET.fromstring, ET.parse, find, findall, findtext, iter, .//tag[@attr="v"], SubElement, indent, tostring. defusedxml for untrusted XML.
Settings configparser: sections, getint, getboolean, fallback=, %(name)s. tomllib.load(binary file) and loads, read only.
pickle dumps and loads any object, Python only, NEVER load untrusted data (__reduce__ runs code).

CSV file size
bytes ≈ rows × (sum of value lengths + separators + 2 for \r\n)

Base64 for binary data in JSON or XML
size out = $4 \times \lceil n / 3 \rceil$

EXAM Always open CSV files with newline="". The csv module handles line endings itself (rows end with \r\n by default), and quotes inside values. Without newline="" you can get blank lines between rows on Windows.

TRAP Never parse XML from strangers with the standard library alone: specially made files can use huge amounts of memory (the "billion laughs" attack). Use the defusedxml package for untrusted XML.

TRAP pickle can save almost any Python object, but pickle.loads can run any code the creator of the data chose, as the Evil class shows with a harmless print. Never unpickle data from the internet, uploads or anyone you do not fully trust. Use JSON for data you exchange.

6.3 Databases with sqlite3 and SQL

Basics sqlite3.connect(path or ":memory:"), execute, executemany, executescript, commit, close, fetchone, fetchall, lastrowid, rowcount.
SQL CREATE TABLE, INSERT, SELECT, UPDATE and DELETE with WHERE, ORDER BY, GROUP BY, HAVING, JOIN, LEFT JOIN, COUNT, SUM, COALESCE, LIMIT.
Safety Values only through ? or :name. Never f-strings in SQL. Check table and column names against a fixed list.
Transactions "with conn:" commits or rolls back (does not close). IntegrityError from PRIMARY KEY, UNIQUE, NOT NULL, CHECK, foreign keys (PRAGMA foreign_keys = ON).
Speed sqlite3.Row, CREATE INDEX, EXPLAIN QUERY PLAN (SCAN vs SEARCH), one commit for many inserts, B-tree height log base f of N.

B-tree height
$h = \lceil \log_f N \rceil$, f = keys per page (fan out)

Selectivity of WHERE a = v
rows ≈ N / V(a), V(a) = number of different values

EXAM The most common beginner bug: INSERT works, the program ends, and the data is gone. You forgot conn.commit(). Using "with conn:" commits for you.

TRAP An UPDATE or DELETE without WHERE changes EVERY row. Write the WHERE first, and try it as a SELECT before you run it.

TRAP Never put values into SQL with f-strings, +, % or .format(). Use ? or :name every time. Placeholders stop SQL injection, handle quotes in names like O'Neil, and let the database reuse the prepared statement.

EXAM Placeholders are for VALUES only. A table or column name cannot be a ?. If a user picks the column to sort by, check it against a fixed list such as {"name", "price"} before you put it into the SQL.

6.4 Desktop windows with tkinter

First window root = tk.Tk(), title, geometry("WxH"), Label, Button(command=f), pack, root.mainloop(). command=f without brackets.

Layout grid(row, column, sticky="nsew", columnspan, padx, pady), columnconfigure(i, weight=1), pack(side, fill, expand), place(x, y). One manager per parent.

Input and variables Entry.get, insert, delete. StringVar, IntVar, BooleanVar, textvariable, variable, trace_add("write", f). messagebox, filedialog.

Events and Canvas bind("", f) with event.x, event.y, event.keysym. create_line, create_oval, move, coords. after(ms, f) for animation.

Responsive apps Callbacks return fast. after() for timers, threads plus queue.Queue for slow work, widgets only from the main thread. Model apart from the view.

Centre a window
x = (screen width − window width) // 2, y = (screen height − window height) // 2

Fitts's law $\text{time} = a + b \times \log_2(D / W + 1)$

EXAM command=say_hello() (with brackets) is the classic beginner bug: it calls the function ONCE while the button is made and stores its result, None, as the command. Then clicks do nothing. To pass arguments, use command=lambda: say(name).

TRAP Never mix pack and grid for widgets with the same parent: Tk raises TclError ("cannot use geometry manager grid inside . which already has slaves managed by pack"). Put each group in its own Frame instead, and each Frame can use its own manager.

TRAP Tk is not thread safe. Create and change widgets only in the thread that runs mainloop. Other threads hand their results over through a queue, and the main thread picks them up with after().

STEP 8

Advanced Python · Exam practice

8.1 Exam practice: PCPP1 style papers

Exam facts PCPP1 (PCPP-32-101) as listed by training providers: 45 questions, 65 minutes, 70 %. Confirm on pythoninstitute.org/pcpp1.

Object traps __eq__ needs __hash__, NotImplemented for unknown types, super() follows the MRO, wraps keeps the name, slots remove __dict__.

Iteration traps Generators run lazily and once, send resumes a yield, yield from returns the inner value, groupby needs sorted input.

Error traps __exit__ True swallows, raise from sets __cause__, except* splits groups, never except Exception: pass.

Library traps Raw strings for regex, json makes lists and str keys, csv gives str, commit() in sqlite3, command=f in tkinter.

TRAP Check pythoninstitute.org/pcpp1 before you book. Exam versions, prices and weights change, and this page could not confirm them from the official site.

EXAM In the exam: read the whole snippet first, then the options. Trace it on paper and watch for the answer "the code raises an exception". Mark hard questions and come back, because every question is worth the same.

STEP 1

Web and APIs · How the web works: HTTP, REST and sockets

1.1 HTTP and REST: how the web talks

Messages Request line (method, path, version), headers, blank line, body. Status line, headers, blank line, body. \r\n line ends.

Methods and codes GET read, POST create (201 + Location), PUT replace, PATCH change, DELETE (204). 2xx ok, 3xx redirect, 4xx client, 5xx server.

URLs urlsplit, parse_qs, parse_qsl, urlencode, quote, urljoin. Percent encoding: one byte becomes %XX.

Headers Content-Type, Accept, Authorization, Set-Cookie (HttpOnly, Secure, SameSite), Cache-Control, ETag and If-None-Match for 304.

Python http.server.BaseHTTPRequestHandler (do_GET, send_response, send_header, end_headers, wfile), urllib.request, HTTPError for 4xx and 5xx.

Time to first byte
about (TCP 1 + TLS 1 + request 1) round trips × RTT

Average time with a cache
average = hit rate × hit time + (1 − hit rate) × miss time

EXAM URLs may only contain a small set of characters. Others, like spaces, é or & inside a value, must be PERCENT ENCODED : each byte becomes % and two hex digits, so a space is %20 and é (2 bytes in UTF-8) is %C3%A9. Use urlencode or let requests do it with params=, never glue query text together by hand.

EXAM urllib.request.urlopen raises HTTPError for 4xx and 5xx answers. The requests library in the next chapters does not: it returns the response, and you call raise_for_status() when you want an exception.

TRAP HTTP itself has no memory between requests. Logins work because the server gives the browser a session cookie or a token, and the client sends it back every time. Mark session cookies HttpOnly, Secure and SameSite, as in the example.

1.2 Sockets, TCP and UDP: under HTTP

Sockets socket(AF_INET, SOCK_STREAM) for TCP, SOCK_DGRAM for UDP. An address is (host, port). Ports 0 to 65,535. 127.0.0.1 local only, 0.0.0.0 every network card.

TCP calls Server: bind, listen, accept (new socket per client). Client: create_connection(addr, timeout). sendall, recv(n) gives 1 to n bytes, b"" means closed.

Framing TCP keeps no message borders. Length prefix with struct.pack("!I", n), or an end marker such as a new line. Loop until a message is complete.

UDP sendto and recvfrom, no handshake, borders kept, may lose or reorder. Used for DNS, games, video. 8 byte header.

Numbers MSS = MTU − 40. throughput ≤ window / RTT. window = bandwidth × RTT. /n network has 2^{32-n} addresses.

Addresses in a network
$\text{addresses} = 2^{32 - \text{prefix}}$, usable = addresses − 2

Window limit on speed
$\text{throughput} \le \text{window} / \text{RTT}$

TCP with loss (Mathis formula)
$\text{throughput} \approx (\text{MSS} / \text{RTT}) \times 1.22 / \sqrt{p}$

EXAM A socket only knows bytes. Turn text into bytes with .encode("utf-8") before sending and back with .decode("utf-8") after receiving. "é" is one letter but 2 bytes, so always measure the length of the bytes, never of the text.

TRAP Host "127.0.0.1" (localhost) only accepts connections from the same computer. "0.0.0.0" accepts them from every network card, so other computers can connect. Use 127.0.0.1 while you learn.

TRAP A very common bug: the sender calls sendall twice and the receiver expects two recv calls to return the two messages. On a fast local test it often works, and then it fails on a real network. recv only promises "some bytes", so always loop until you have a whole message.

EXAM A /24 network has $2^{32-24} = 256$ addresses. The first is the network address and the last is the broadcast address, so 254 are left for computers. Ports go from 0 to 65,535, and ports below 1024 usually need admin rights.

STEP 2

Web and APIs · Calling web APIs

2.1 Calling web APIs with requests

Calls requests.get, post, put, patch, delete(url, params=, json=, data=, headers=, timeout=). Response: status_code, ok, headers, text, content, json().

Errors raise_for_status() for 4xx and 5xx. HTTPError, ConnectionError, Timeout, JSONDecodeError, all under RequestException (catch it last).

Timeouts No default timeout. timeout=5 or timeout=(connect, read). Always pass one.

Session Reuses connections, keeps cookies and headers. Auth: Authorization: Bearer token, auth=(user, pw), API key header. Secrets come from environment variables.

JSON object to dict, array to list, null to None. [] for required keys, .get() for optional. datetime.fromisoformat for dates.

Pages to fetch $\text{pages} = \lceil \text{items} / \text{page size} \rceil$

Time with a Session
(handshake RTTs + n) × RTT

Chance one of k tries works $P = 1 - (1 - p)^k$

TRAP Always pass TIMEOUT= . requests has no timeout by default, so a server that never answers keeps your program waiting for ever. A number such as timeout=5 is used for connecting and again for each wait while reading.

EXAM A SESSION keeps cookies and default headers, and it reuses the TCP (and TLS) connection for every request to the same host. The example shows 3 requests over 1 connection. That saves one or two round trips each time, so use a Session whenever you make more than a couple of calls.

TRAP Never put a real token or password in code that you share or commit to git. Read it with os.environ["API_TOKEN"] and keep it in a .env file that git ignores.

2.2 Real API work: pages, retries, limits and validation

Pages Loop with a generator until no next page or cursor. Cursors stay correct when data changes. pages = ⌈items / size⌉.

Retries Only timeouts, connection errors, 429, 502, 503, 504, and only idempotent requests. wait = random(0, min(cap, base × 2^n)). Stop after k tries.

Limits 429 plus Retry-After (seconds or a date). Token bucket: capacity for bursts, rate for the average. X-RateLimit-Remaining.

pydantic class M(BaseModel), Field(min_length=, ge=, le=), M.model_validate(dict), model_dump(), TypeAdapter(list[M]).validate_json(text), ValidationError.errors().

httpx httpx.Client like a Session, httpx.AsyncClient with await, asyncio.gather for many, asyncio.Semaphore(n) to limit them.

Backoff wait $\text{wait}_n = \min(\text{cap}, \text{base} \times 2^n)$

Token bucket in time T
at most capacity + rate × T calls

Calls in flight (Little's law)
$L = \text{rate} \times \text{time per call}$

EXAM A token bucket holds up to capacity tokens and refills rate tokens every second. Each call takes one token, and with no token the call must wait or gets 429. So it allows short bursts but keeps the average at the rate. Servers use it to protect themselves, and clients can use it to stay polite.

STEP 3

Web and APIs · Flask: small and flexible web apps

3.1 Flask basics: routes, templates, forms and JSON

Routes @app.route(rule, methods=[...]) or @app.get / post. Converters string, int, float, path, uuid. url_for(endpoint, **values). 404 no match, 405 wrong method.

Request request.args.get(name, default, type=int), request.form, request.get_json(silent=True), request.files, headers, cookies.

Responses str is HTML, dict or list is JSON, (body, status, headers), redirect(url_for(...)), abort(code), make_response, @app.errorhandler(404).

Templates and sessions render_template, {{ }} escaped, {% for %}, filters, extends and block. session is a signed cookie, needs secret_key. flash and get_flashed_messages.

Run and test app.test_client() for tests. flask --app app run --debug only on your computer. gunicorn -w 4 "app:app" in production.

Requests in progress (Little's law) $L = \lambda \times W$

Throughput of sync workers
requests/s ≤ workers / time per request

EXAM A route answers only GET (and HEAD, OPTIONS) unless you pass methods=. A wrong method gives 405 METHOD NOT ALLOWED , a URL that matches no rule gives 404 NOT FOUND .

TRAP Jinja ESCAPES every value by default in .html templates, so a name such as <script> is shown as text and never runs. This stops cross site scripting (XSS). Only mark text as safe with |safe when you made it yourself.

TRAP The session cookie is signed, NOT encrypted: the example decodes it and reads the user name. Never put secrets in it, keep secret_key secret and long, and read it from an environment variable.

EXAM Real forms also need protection against cross site request forgery (CSRF), so another website cannot post forms as your user. The extension Flask-WTF adds a hidden token to every form and checks it.

TRAP A global dict like BOOKS is fine for learning, but it is lost when the server restarts, and each Gunicorn worker process has its own copy. Real apps keep data in a database, as the next chapter shows.

3.2 Real Flask apps: structure, database, login and tests

Structure create_app(settings) returns a new app. Blueprint(name, __name__, url_prefix=...), app.register_blueprint(bp). url_for("bp.view"). current_app, not a global app.

Database get_db() on g, sqlite3.Row, ? placeholders, commit(), app.teardown_appcontext(close_db), @app.cli.command("init-db").

Auth generate_password_hash / check_password_hash (salted, slow). session.clear() then session["user_id"]. before_app_request loads g.user. login_required with functools.wraps.

Tests pytest fixture in conftest.py builds create_app({"TESTING": True}) and returns app.test_client(). app.test_cli_runner() for commands.

Production Config from environment (from_prefixed_env), app.logger, errorhandler passing HTTPException through, gunicorn "shop:create_app()" behind HTTPS.

Guesses per second against a slow hash
guesses/s = raw hashes per second / iterations

Average time to guess
time ≈ N / 2 / guesses per second

EXAM Inside a blueprint, link with the blueprint name: url_for("api.status") or url_for(".status") for the same blueprint. Use CURRENT_APP instead of a global app variable, because with a factory there is no global app.

EXAM For bigger apps, Flask-Login manages the logged in user and "remember me", and Flask-WTF adds CSRF tokens to forms. They do the same jobs as the code here, with more cases covered.

STEP 4

Web and APIs · Django: the framework with everything built in

4.1 Django basics: project, URLs, views, templates, models and admin

Project django-admin startproject mysite, python manage.py startapp books, add "books" to INSTALLED_APPS. manage.py runserver, makemigrations, migrate, createsuperuser, shell, check --deploy.

URLs and views urlpatterns = [path("books/<int:book_id>/", views.book_detail, name="book-detail")], include("books.urls"). A view takes request and returns HttpResponse or JsonResponse. Http404, redirect, reverse.

Templates render(request, "books/list.html", context). {{ value|filter }}, {% for %}{% empty %}{% endfor %}, {% if %}, {% url "name" arg %}, {% extends %} with {% block %}, {% csrf_token %} in POST forms. Values are escaped.

Models class Book(models.Model) with CharField, DecimalField, ForeignKey(on_delete=...). Book.objects.create / all / filter(year__gte=1900) / get / update / delete. related_name for the other side.

Admin @admin.register(Book) class BookAdmin(admin.ModelAdmin) with list_display, list_filter, search_fields. path("admin/", admin.site.urls). Only staff users can log in.

Number of pages pages = ⌈ n / per page ⌉

OFFSET of a page
OFFSET = (page − 1) × per page

The N + 1 problem queries = 1 + n

EXAM Django calls its pattern MTV : the MODEL holds the data, the TEMPLATE shows it, and the VIEW decides what to show. It is the same idea as MVC, only the names differ: the Django view does the job of an MVC controller.

EXAM The test client does not check CSRF tokens unless you ask it to, so the POST in the example works without one. In a real project CsrfViewMiddleware is on, and every POST form needs {% csrf_token %}. The animation above shows what happens without it.

EXAM In a project you never call schema_editor yourself. You change models.py, run makemigrations to write a migration file, and migrate to run it. Chapter 4.2 goes much deeper into the ORM and migrations.

4.2 Django in depth: ORM, migrations, forms, users and tests

QuerySets Lazy until read. filter, exclude, order_by, get, first, exists, count. Lookups: __gte, __icontains, __in, author__name. Q(a) | Q(b), ~Q(c). F("col") inside filter and update.

Aggregation and speed aggregate(Avg("price")) gives a dict, annotate(n=Count("books")) adds n per row. select_related for ForeignKey, prefetch_related for ManyToMany. only, values_list, bulk_create, indexes.

Migrations makemigrations then migrate. Commit the files. New fields need a default or null=True. Data migrations use RunPython with apps.get_model. migrate app 0001 goes back. sqlmigrate shows the SQL.

Forms form = BookForm(request.POST), form.is_valid(), cleaned_data, form.save(). clean_<field> for one field, clean() with add_error for several. ModelForm with Meta.model and Meta.fields.

Users, transactions, tests create_user, check_password, authenticate, login_required, permission_required, Group. transaction.atomic() all or nothing. TestCase rolls back each test, setUpTestData, self.client, assertNumQueries.

Queries in a loop
N + 1 without help, 1 with select_related, 2 with prefetch_related

Time of a page
time ≈ queries × round trip + work

Search with a B-tree index steps ≈ $\log_b(n)$

EXAM In tests, assertNumQueries(n) fails when a change adds queries, so an N + 1 problem cannot sneak back in. In development, django-debug-toolbar shows every query of a page.

EXAM The usual view: if request.method == "POST", bind and check the form, save and redirect when it is valid. Otherwise make an empty form. Either way render the template, which shows the errors. In the template put {% csrf_token %} inside the form tag.

EXAM login_required sends a person who is not logged in to LOGIN_URL with ?next=, as the 302 above shows. permission_required with raise_exception=True gives 403 instead, which is right for a logged in user who simply may not do this.

EXAM Look at the query count: inside a test the atomic() of transfer becomes SAVEPOINT and RELEASE, because TestCase already opened a transaction around the test. Outside tests it is BEGIN and COMMIT.

STEP 5

Web and APIs · Building APIs: Django REST framework and FastAPI

5.1 Django REST framework: JSON APIs on top of Django

Serializers class BookSerializer(serializers.ModelSerializer) with Meta.model, Meta.fields. serializer.data, Serializer(data=...).is_valid(raise_exception=True), validated_data, save(). validate_<field>, validate, SerializerMethodField, read_only.

Views and routers @api_view(["GET", "POST"]) with Response(data, status=201). class BookViewSet(viewsets.ModelViewSet) with queryset and serializer_class. router.register("books", BookViewSet). @action. get_queryset, perform_create.

Security DEFAULT_AUTHENTICATION_CLASSES (Token, Session, JWT). permission_classes: IsAuthenticated, IsAuthenticatedOrReadOnly, has_object_permission. 401 no credentials, 403 not allowed. Throttles answer 429 with Retry-After.

Lists DEFAULT_PAGINATION_CLASS PageNumberPagination with PAGE_SIZE: count, next, previous, results. SearchFilter ?search=, OrderingFilter ?ordering=-year. Always order the queryset. CursorPagination for feeds.

Tests and design APITestCase, self.client.post(url, data, format="json"), force_authenticate, assertNumQueries. Nouns in URLs, right status codes, one error shape, versioning, idempotent retries.

Number of pages
pages = ⌈ count / page size ⌉

Requests allowed rate = requests / window

Wait time in a sliding window
Retry-After = oldest + window − now

EXAM SearchFilter and OrderingFilter come with DRF. For filters such as ?year=1965&author=2, the django-filter add on reads them from a filterset_fields list. Always order the queryset, otherwise pages can show a row twice.

5.2 FastAPI: fast, typed APIs with automatic docs

Endpoints @app.get("/books/{book_id}") def get_book(book_id: int). Query parameters are plain parameters, Query(10, le=100) adds limits. A Pydantic model parameter is the body. response_model, status_code, HTTPException(404, "...").

Pydantic class BookIn(BaseModel) with Field(gt=0), Field(min_length=...), Literal, nested models, @field_validator, @model_validator. model_dump(), model_dump_json(), model_json_schema(). Bad input is a 422 with locations.

Dependencies Depends(get_db) and Annotated[T, Depends(f)]. yield for clean up. Header() for tokens, OAuth2PasswordBearer for logins. app.dependency_overrides in tests.

Async async def only with awaitable libraries (httpx.AsyncClient, asyncpg). Plain def for blocking code, which runs in a thread pool. Never time.sleep or requests.get in async def. BackgroundTasks, lifespan.

Run and test fastapi dev main.py, fastapi run or uvicorn main:app --workers 4. /openapi.json, /docs, /redoc. TestClient(app) with pytest fixtures and parametrize.

Time with await
total ≈ the longest single wait

Time with a blocked loop total ≈ n × wait

Requests one process can hold (Little's law)
in flight = rate × time per request

EXAM Run the real server with `fastapi dev main.py` while you develop (it reloads on every save) and `fastapi run main.py` or `uvicorn main:app --workers 4` in production. Then open http://127.0.0.1:8000/docs.

STEP 6

Web and APIs · Securing APIs and the web revision exam

6.1 API security: passwords, JWT, OAuth 2.0, CORS and OWASP

Passwords hashlib.pbkdf2_hmac("sha256", pw, salt, 600_000) or Argon2id. Random salt per user, stored as algorithm$rounds$salt$hash. hmac.compare_digest. secrets.token_urlsafe(32), never random. Keys from os.environ, never in git.

JWT header.payload.signature in base64url, HS256 = HMAC-SHA256. Readable by anyone. jwt.encode(claims, key, algorithm="HS256"). jwt.decode(token, key, algorithms=["HS256"], audience=..., issuer=...). Claims sub, exp, iat, nbf, aud, iss, jti. Short access, rotated refresh, kid for keys.

FastAPI auth OAuth2PasswordBearer(tokenUrl="token", scopes=...), OAuth2PasswordRequestForm, Security(current_user, scopes=[...]). 401 with WWW-Authenticate: Bearer, 403 for missing scope, 404 for another user's object.

Browser rules CORSMiddleware(allow_origins=[exact list], allow_credentials=True, allow_methods, allow_headers). Preflight OPTIONS. Never * with credentials. CSRF: SameSite cookies and CSRF tokens. Headers: HSTS, nosniff, Content-Security-Policy.

OWASP API Top 10 API1 object level checks, API2 authentication, API3 property level (mass assignment, extra="forbid"), API4 resource limits, API5 function level, API7 SSRF. ? placeholders against injection. Rate limits with 429 and Retry-After.

Entropy of a random secret
bits = length × $\log_2$(alphabet size)

Guesses to try every value guesses = 2^{bits}

Time to crack
time = guesses × cost per guess / machines

EXAM Keep secret keys out of the code: read them with `os.environ["SECRET_KEY"]`, keep a local .env file in .gitignore, and use a vault (AWS Secrets Manager, HashiCorp Vault) in production. A key that was ever pushed to git is burnt: rotate it, because deleting the commit does not remove copies.

6.2 Web revision exam: HTTP, Flask, Django, FastAPI and security

HTTP Status classes 2xx, 3xx, 4xx, 5xx. 201 with Location, 204, 301, 304, 400, 401, 403, 404, 405, 412, 422, 429. Idempotent: GET, PUT, DELETE. requests.get(url, timeout=5) and raise_for_status().

Flask @app.get("/books/<int:book_id>"), request.args.get("page", 1, type=int), request.get_json(), return dict, status, headers. g, session, Blueprint(url_prefix=...), url_for, errorhandler, test_client().

Django and DRF Lazy QuerySets, Q, F, annotate, select_related and prefetch_related, makemigrations and migrate, reverse and resolve, autoescape. Serializer.is_valid() then validated_data, ModelViewSet with a router.

FastAPI Type hints decide path, query and body. Pydantic gives 422 with locations. response_model, Depends with yield, async def only with await, TestClient and dependency_overrides.

Security Slow salted hashes, JWT with pinned algorithms and short exp, scopes give 403, owner checks give 404, CORS for browsers, ? placeholders, caps on page size, secrets in the environment.

Codes available $\text{codes} = \text{alphabet}^{\text{length}}$

EXAM In a written exam, trace each snippet on paper before you look at the options, and watch for the answers "404", "405" and "the code raises". In an interview, say your assumptions out loud, start with the simplest working API, then add caching, security and limits.